

SAE302_Documentation_Technique

Réponse au cahier des charges

Ce que le projet demandait

Le projet SAE 302 demandait de réaliser un système de communication anonymisée inspiré du réseau TOR, comprenant :

- Un **Master** centralisant les routeurs
- Plusieurs **routeurs** effectuant un routage multi-sauts
- Des **clients** capables d'envoyer et recevoir des messages
- Un **chiffrement en couches (oignon)**
- Une **base de données MariaDB**
- Une **utilisation de sockets réseau**
- Un **déploiement sur plusieurs machines virtuelles**
- Une **interface graphique** pour les clients (et éventuellement le master)

Ce que j'ai réussi à faire

Les éléments suivants ont été **entièrement implémentés et fonctionnels** :

- Routage multi-sauts entre clients
- Chiffrement en couches (oignon)
- Génération de clés RSA par les routeurs
- Enregistrement automatique des routeurs auprès du Master
- Stockage des routeurs dans une base MariaDB
- Clients bidirectionnels (peuvent envoyer et recevoir)
- Interface graphique pour les clients (PyQt)
- Déploiement sur :

1 VM Linux (Master + BDD)

1 VM Linux (Routeurs)

1 VM Windows (Clients)

Interface graphique du Master

Interface graphique du clientA

Interface graphique du clientB

2. Explication du projet

Structure du code

src/

client/

clientA.py

clientB.py

client_gui.py

router/

router_server.py

launch_routers.py

master/

master_server.py

master_gui.py

common/

crypto.py

onion.py

sockets.py

database.py

Fonctionnement des composants

Master

- Écoute sur un port TCP
- Reçoit les routeurs
- Stocke leurs informations dans MariaDB
- Fournit la liste des routeurs aux clients

Routeurs

- Génèrent une clé RSA
- S'enregistrent auprès du Master
- Déchiffrent une seule couche du message
- Redirigent vers le prochain saut

Clients

- Demandent la liste des routeurs
- Choisissent un chemin aléatoire
- Chiffrent le message en couches
- Peuvent envoyer ou recevoir des messages

Protocole / échanges réseau

- TCP uniquement
- Messages JSON ou chaînes structurées
- Commandes simples (REGISTER, GET_ROUTERS)
- Communication directe entre routeurs

3. Chiffrement

Fonctionnement

- Chaque routeur possède une clé RSA
- Le client chiffre le message couche par couche
- Chaque routeur déchiffre uniquement sa couche

Forces

- Respect du principe du routage en oignon
- Aucun routeur ne connaît l'origine complète
- Fonctionnement clair et pédagogique

Faiblesses (important pour la note)

- RSA simplifié (petits nombres premiers)
- Pas de padding cryptographique
- Vulnérable en conditions réelles

- Sécurité non adaptée à la production

Ces choix sont **volontaires et assumés** dans un cadre pédagogique.